

Documentation technique — EcoRide

TP Développeur Web et Web Mobile (Studi) — Auteur : Evan Orofino — 2026

1. Réflexions initiales et choix technologiques

1.1 Contexte

EcoRide est une plateforme de covoiturage écologique avec 13 user stories à implémenter, devant gérer :

- Une **base relationnelle** (utilisateurs, voitures, trajets, participations)
- Une **base NoSQL** (avis, préférences, configuration)
- Un **front-end responsive**
- Un **back-end sécurisé**
- Un **déploiement** documenté

1.2 Stack retenue

Couche	Choix	Pourquoi
Front	HTML5 + CSS3 + JS vanilla	Pas de framework lourd, contrôle total, performances maximales, simple à défendre à l'oral
CSS	Custom + variables CSS	Charte graphique propre sans dépendance, mobile-first
Icons	Bootstrap Icons (CDN)	Léger, cohérent, large catalogue
Charts	Chart.js (CDN)	Standard de fait, simple à intégrer
Back	PHP 8.2 + PDO	Recommandé Studi, large communauté, déploiement universel
BDD relationnelle	MySQL 8 / MariaDB 10	Mature, performant, bien intégré à PHP
BDD NoSQL	MongoDB	Schéma flexible idéal pour avis et préférences

Couche	Choix	Pourquoi
Templating	PHP natif (vues)	Pas de couche supplémentaire à apprendre/défendre
Autoloading	Composer PSR-4	Standard PHP, classes auto-chargées
Déploiement	Railway.app	Plan gratuit, déploiement Git-push

1.3 Alternatives considérées et écartées

Alternative	Pourquoi écartée
Framework Symfony / Laravel	Surdimensionné pour 13 US, ajoute une couche d'apprentissage masquant les concepts
Node.js / Express	Possible, mais PHP est explicitement mentionné dans la consigne et plus aligné avec la formation DWWM
React / Vue côté front	Inutile pour un site avec 90 % de contenu statique côté serveur ; SEO et perfs initiales meilleures avec SSR PHP
Bootstrap CSS	Custom CSS plus léger et cohérent avec la charte ; on garde Bootstrap Icons uniquement

2. Configuration de l'environnement

2.1 Prérequis

- Windows 10/11 (testé) ou Linux/macOS
- XAMPP 8.2 (ou PHP 8.2 + MySQL 8 séparés)
- Composer 2.x
- Git 2.x
- Extension PHP `mongodb` (driver PECL)
- Compte GitHub (dépôt public)
- Compte MongoDB Atlas (cluster gratuit M0)

2.2 Installation pas à pas

```
# 1. Cloner le dépôt
git clone https://github.com/<votre-user>/ecoride.git
cd ecoride

# 2. Configurer l'environnement
cp .env.example .env
# Éditer .env : DB_*, MONGO_URI, APP_SECRET
```

3. Installer les dépendances PHP

```
composer install
```

4. Créer et peupler MySQL

```
mysql -u root < sql/01_schema.sql
```

```
mysql -u root < sql/02_seed.sql
```

5. Initialiser MongoDB

```
php sql/init_mongo.php
```

6. Lancer le serveur de dev

```
php -S localhost:8000 -t public
```

2.3 Structure du projet

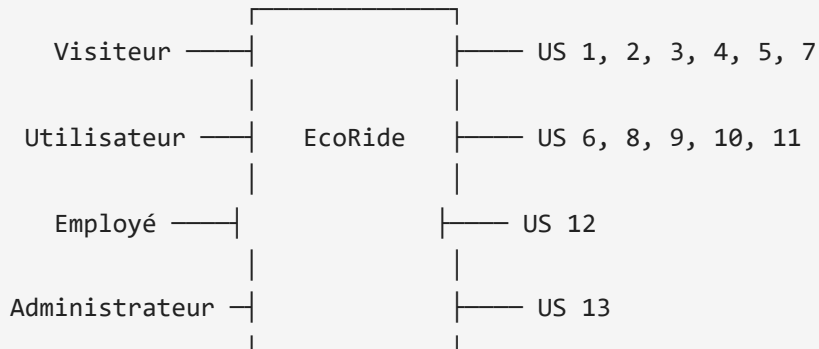
```
ecoride/
├── config/                # Configuration (BDD, Mongo, app)
│   └── config.php
├── public/               # Document root (servi par Apache/PHP)
│   ├── index.php        # Front controller (toutes les requêtes)
│   ├── .htaccess        # URL rewriting + headers sécurité
│   ├── css/app.css
│   ├── js/app.js
│   └── images/
├── src/                  # Code applicatif (autoload PSR-4 → App\ )
│   ├── Core/            # Database, Mongo, Router, Controller, Auth, Security
│   ├── Controllers/     # Logique HTTP (par domaine fonctionnel)
│   ├── Models/          # Accès aux données SQL et Mongo
│   └── Helpers/functions.php
├── views/                # Templates PHP (un dossier par domaine)
│   ├── layouts/main.php
│   ├── partials/
│   └── home/, auth/, covoiturage/, user/, employe/, admin/, errors/
├── sql/
│   ├── 01_schema.sql    # DDL MySQL
│   ├── 02_seed.sql      # Données de test
│   └── init_mongo.php    # Init Mongo
├── docs/                 # Documentation (ce fichier inclus)
├── routes.php            # Définition des routes
├── composer.json
├── .env.example
└── README.md
```

3. Architecture logicielle

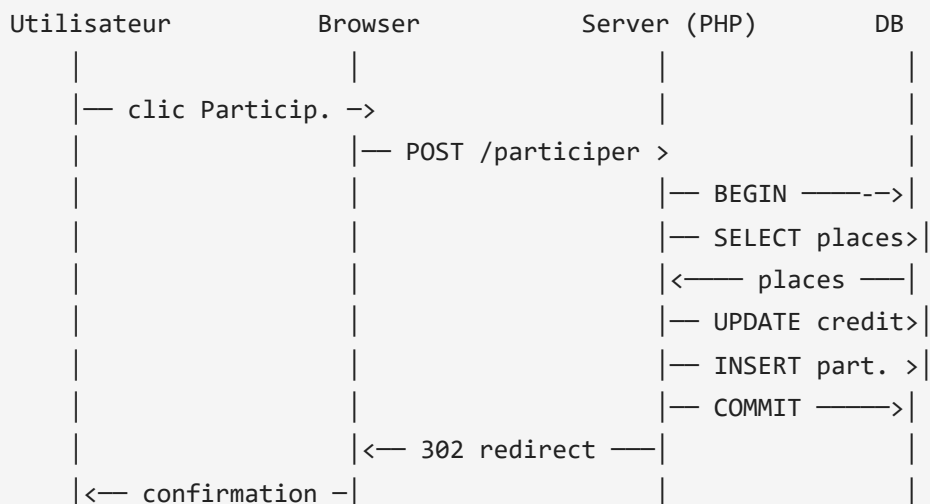
3.1 Pattern MVC simplifié

- **Front controller** (`public/index.php`) : point d'entrée unique, démarrage session, dispatching
- **Router** (`src/Core/Router.php`) : map URI → controller@method
- **Controllers** : reçoivent les inputs HTTP, valident, appellent les Models, rendent une vue
- **Models** : encapsulent les requêtes SQL/Mongo (jamais de SQL dans les controllers)
- **Views** : templates PHP avec helpers (`e()` , `csrf_field()` , `format_date_fr()`)

3.2 Diagramme d'utilisation (Use Case)



3.3 Diagramme de séquence (US 6 — Participer à un covoiturage)



4. Modèle de données

Voir `mcd.md` pour le diagramme complet et `schema_mongodb.md` pour les collections NoSQL.

4.1 Répartition SQL / NoSQL

Donnée	Stockage	Justification
Utilisateurs, rôles, voitures, covoiturages, participations, crédits	MySQL	Transactions ACID, intégrité référentielle
Avis	MongoDB	Modération asynchrone, lecture massive
Préférences chauffeur	MongoDB	Schéma extensible (clés libres)
Configuration globale	MongoDB	Lectures fréquentes, écritures rares
Logs applicatifs	MongoDB	Append-only, pas de relations

5. Sécurité

5.1 Authentification

- Mots de passe hachés en **bcrypt** (`password_hash` / `password_verify`)
- **Validation de robustesse** côté serveur (`Security::validatePasswordStrength`) : min 8 caractères, 1 majuscule, 1 minuscule, 1 chiffre, 1 spécial
- **Régénération du `session_id`** à la connexion (`session_regenerate_id`)

5.2 Sessions

```
ini_set('session.cookie_httponly', '1');  
ini_set('session.cookie_samesite', 'Lax');  
ini_set('session.use_strict_mode', '1');
```

5.3 CSRF

- Token aléatoire 32 octets stocké en session
- Champ caché `_csrf` dans tous les formulaires POST
- Vérification par `hash_equals` (timing-safe)

5.4 Injection SQL

- Toutes les requêtes utilisent **PDO en mode prepared statements** avec paramètres nommés
- `PDO::ATTR_EMULATE_PREPARES = false` (vraies requêtes préparées côté serveur)

5.5 XSS

- Échappement systématique via `e()` (alias `htmlspecialchars`)
- Header `X-XSS-Protection: 1; mode=block`
- Header `X-Content-Type-Options: nosniff`

5.6 Clickjacking

- Header `X-Frame-Options: SAMEORIGIN`

5.7 Mots de passe en BDD

- Aucun mot de passe en clair, jamais loggé
- Connexion DB protégée par `.env` exclu du dépôt

5.8 Contrôle d'accès

- `Auth::requireLogin()` : redirige vers `/login` si non authentifié
- `Auth::requireRole($roles)` : 403 si l'utilisateur n'a pas le rôle requis
- Vérification systématique de propriété (ex : un user ne peut annuler que ses trajets)

6. Performance

- Requêtes SQL avec **JOIN** plutôt que N+1
- **Index** sur les colonnes filtrées (lieu_depart, lieu_arrivee, date_depart, statut)
- **Cache navigateur** des assets statiques (CSS, JS, images) via `.htaccess`
- **Compression gzip** activée pour HTML/CSS/JS
- Connexions BDD en singleton (une seule instance PDO/MongoDB par requête)

7. Déploiement

7.1 Cible

Railway.app (alternative gratuite et plus simple que Heroku en 2026).

7.2 Étapes

1. Pousser le dépôt sur GitHub (public)
2. Sur Railway : `New Project > Deploy from GitHub`
3. Ajouter une **variable d'environnement** : `DATABASE_URL` (Railway génère MySQL)
4. Ajouter la **MONGO_URI** depuis MongoDB Atlas

5. Configurer le **build command** (Railway détecte PHP automatiquement)
6. Le `composer install` est lancé à chaque push

7.3 Fichier `Procfile` (à créer)

```
web: vendor/bin/heroku-php-apache2 public/
```

7.4 Variables d'environnement attendues sur Railway

Variable	Source
<code>APP_ENV</code>	<code>production</code>
<code>APP_DEBUG</code>	<code>false</code>
<code>APP_URL</code>	URL Railway générée
<code>APP_SECRET</code>	32 caractères aléatoires
<code>DB_HOST</code> , <code>DB_NAME</code> , <code>DB_USER</code> , <code>DB_PASSWORD</code>	Plugin Railway MySQL
<code>MONGO_URI</code> , <code>MONGO_DB</code>	MongoDB Atlas connection string

8. Évolutions possibles

- Ajout d'un **système de messagerie** entre chauffeur et passager
- **API REST** pour application mobile native
- **Géolocalisation** pour suggérer des trajets sur la route du chauffeur
- **Paiement Stripe** pour acheter des crédits
- **Notifications push** (PWA + service worker)
- Intégration d'un **calculateur d'empreinte carbone** par trajet